Lecture 2-7 — Event Types & Input Events

This note explains common keyboard/input events: `keydown`, `keypress`, `keyup`, `input`, and `change`. Examples show when each fires and how to listen safely.

Code example

```
var myInp = document.querySelector("#myInp");
myInp.addEventListener("change", () => console.log("change fired"));
myInp.addEventListener("keydown", ev => console.log("keydown", ev.key));
myInp.addEventListener("keypress", ev => console.log("keypress", ev.key));
myInp.addEventListener("keyup", ev => console.log("keyup", ev.key));
myInp.addEventListener("input", ev => console.log("input", ev.data));
```

Interactive example

Type in the box below to see events:

Output:

```
No events yet.
```

Explanation (for beginners)

- **`keydown`**: fires when a key is pressed down (before the character appears).
- **`keypress`**: fires while a key that produces a character is pressed — deprecated, avoid using it.
- **`keyup`**: fires when the key is released.
- **`input`**: fires whenever the element's value changes (typing, paste, speech, autocomplete).
- **`change`**: fires when the value is committed (e.g., input loses focus after edit).

Powered by SPRK Technologies

Detailed behavior & practical tips

Event ordering example when typing a character key: `keydown → (keypress) → input → keyup → change` (change occurs later when input loses focus). Modern code should use `keydown`/`keyup` and `input` rather than `keypress`.

Which properties to read from keyboard events

- **`ev.key`**: the character produced or the descriptive key name (e.g., `'a'`, `'Enter'`).
- **`ev.code`**: the physical key on the keyboard (e.g., `'KeyA'`, `'Digit1'`), useful for layout-independent detection.
- **Avoid** `keyCode` — it's deprecated and inconsistent across platforms.

Stopping propagation & default behavior

Sometimes you need to prevent the browser's default action (e.g., prevent form submit on Enter) or stop the event from bubbling to parent handlers:

```
element.addEventListener('keydown', ev => {
  if(ev.key === 'Enter'){
    ev.preventDefault(); // don't submit
    ev.stopPropagation(); // don't let parent handlers run
  }
});
```

Debounce vs throttle

When handling frequent events (like `input` or `keyup`) you often need to reduce how often your heavy work runs. Two common techniques:

Debounce (wait until user stops)

Analogy: you're typing a search query — you want to run the search only after the user pauses typing. Debounce waits for a pause and then runs the function once.

1. Reset a timer on each event.

Powered by SPRK Technologies

2. If no new event arrives within the wait time, call the function once.

Annotated debounce example (easy to follow):

```
// debounce: return a wrapper that delays calling `fn`
function debounce(fn, wait){
  // `timeoutId` stores the scheduled call
  let timeoutId = null;

  return function(...args){
    // if a call is already scheduled, cancel it
    if(timeoutId) clearTimeout(timeoutId);

    // schedule fn to run after `wait` milliseconds
    timeoutId = setTimeout(() => {
      fn.apply(this, args); // call original function with same `this` and
arguments
      timeoutId = null; // clear the id
    }, wait);
  };
}

// usage: only call `search()` after user stops typing for 400ms
const debouncedSearch = debounce(function(e){
  // `e.target.value` contains the latest text
  console.log('Search for:', e.target.value);
}, 400);
input.addEventListener('input', debouncedSearch);
```

Step-by-step flow (typing quickly):

1. User types 'a' → timer set for 400ms.
2. User types 'b' before 400ms → previous timer cleared, new timer set.
3. When user stops typing and 400ms pass → function runs once with the final value.

Powered by SPRK Technologies

Throttle (limit frequency)

Analogy: a gate that only lets one person through per second. Throttle ensures a function runs at most once in a given time window.

1. Allow the first event to run immediately.
2. Ignore or defer events until the time window passes.

Annotated throttle example:

```
// throttle: return a wrapper that ensures `fn` runs at most once per
`limit` ms
function throttle(fn, limit){
  let lastCall = 0; // timestamp of last allowed call

  return function(...args){
    const now = Date.now();
    if(now - lastCall >= limit){
      lastCall = now;
      fn.apply(this, args);
    }
    // otherwise ignore this event
  };
}

// usage: handle scroll/input at most once every 200ms
const throttledHandler = throttle(function(e){
  console.log('Handled at', new Date().toISOString());
}, 200);
input.addEventListener('input', throttledHandler);
```

When to use which:

- **Debounce**: when you want the action after the user stops (search box, form validation finishing).
- **Throttle**: when you want a steady rate of updates (scroll/resize handlers, pointermove logging).

Both patterns prevent excessive work and make interfaces feel smoother.

Powered by SPRK Technologies

Common beginner mistakes

- Using `keypress` for modern code — prefer `keydown`/`keyup`.
- Reading `ev.keyCode` — it's legacy.
- Not considering composition/input methods — use `input` for value changes from IME, paste, or speech.

Small exercises

1. Log both `ev.key` and `ev.code` while typing and explain the difference.
2. Implement a search box that sends a search request only after the user stops typing for 400ms (use debounce).

Powered by SPRK Technologies